# ДЗ 3. Аналитика вакансий с RustFS и DuckDB

Карпов Данил Иванович, группа J3310
Индивидуальный вариант: ML, RnD. Эксперимент: 11 октября 2026 года.

## 1. Цель и источник данных

Построен воспроизводимый пайплайн: отбор вакансий своего варианта, загрузка в объектное хранилище, независимый подсчёт частот средствами MapReduce и SQL, полная сверка результатов, профилирование, анализ зарплат и трёх регионов.

Использован открытый набор Ilya (ilyazawilsiv) «IT Vacancies from HeadHunter Website», Kaggle, версия 1, Apache 2.0 [1]. Период исходных публикаций: 18.09-17.10.2023. Файл содержит вакансии России и других стран присутствия HeadHunter. Это историческая выборка, а не состояние рынка в октябре 2026 года.

| Показатель | Значение |
|---|---|
| Исходный CSV | 68 770 записей; 294 284 813 байт |
| Вакансии ML/RnD | 710 |
| С указанными навыками | 422 (59.4%) |
| С указанной зарплатой в любой валюте | 118 (16.6%) |
| Повторяющиеся id | 0 |
| Объект full.jsonl | 217 135 байт; UTF-8, одна вакансия на строку |

Отбор включает профессиональную роль «Дата-сайентист» либо названия с ML, AI, Data Science, Machine/Deep Learning, R&D/RnD, Research Engineer/Scientist, машинным обучением, нейронными сетями или компьютерным зрением. Это явное воспроизводимое правило для указанного пользователем сегмента; официальная готовая выгрузка варианта не предоставлялась.

Навыки очищены от пробелов по краям и приведены к единому регистру Unicode casefold. Синонимы не объединяются: «machine learning» и «машинное обучение» остаются разными исходными метками. Отсутствие списка навыков не считается доказательством отсутствия навыков.

В ДЗ 2 использовался другой CSV без зарплат. Здесь повторно применён проверенный бинарный файл RustFS 1.0.1, а данные и хранилище ДЗ 3 изолированы от предыдущего опыта.

# 2. Хранилище и независимые расчёты

Стенд: macOS-26.3.1-arm64-arm-64bit; Python 3.12.14, DuckDB 1.5.6. RustFS 1.0.1 запущен на 127.0.0.1:19010, bucket hw3-ml-rnd. Использованы четыре локальные папки и EC:2; это один компьютер и один SSD. Физическая отказоустойчивость и распределённый кластер этим опытом не проверяются.

Загружены pilot.jsonl (100 вакансий) и full.jsonl (710 вакансий). Оба объекта прочитаны обратно через GetObject; размеры и SHA-256 совпали с локальными исходниками. Перед и после серии замеров проверяется неизменность ETag объекта.

## Собственная реализация MapReduce

S3 GetObject читается блоками по 65 536 байт. Незавершённая JSON-строка переносится в следующий блок, поэтому граница чанка может проходить внутри UTF-8 символа. Map получает до 250 вакансий, удаляет повторные навыки внутри каждой вакансии и выдаёт счётчики неупорядоченных сочетаний размеров 1, 2 и 3. Reduce суммирует локальные счётчики. Для двух процессов одновременно ожидаются не более четырёх map-задач.

Сложность по времени пропорциональна сумме C(k,1)+C(k,2)+C(k,3), где k - число уникальных навыков вакансии. Чтение входа ограничено чанками и пакетами, но итоговый словарь занимает память пропорционально числу разных сочетаний. Этот вариант не обещает постоянного расхода памяти.

## Независимый SQL в DuckDB

Расширение httpfs и S3 secret дают прямое чтение s3://hw3-ml-rnd/full.jsonl [2]. DuckDB самостоятельно разворачивает списки навыков и строит сочетания соединениями по id; частоты считаются GROUP BY. SQL не использует результат Python MapReduce.

```
SELECT a.id, [a.skill, b.skill, c.skill] AS skills
FROM skills a
JOIN skills b ON a.id = b.id AND a.skill < b.skill
JOIN skills c ON a.id = c.id AND b.skill < c.skill;

SELECT size, skills, count(*)
FROM combinations
GROUP BY ALL;
```

| Полная сверка | Навыки | Пары | Тройки |
|---|---|---|---|
| Пилот: 100 вакансий | 132 | 915 | 3495 |
| Полный набор: 710 вакансий | 587 | 5 811 | 27 058 |

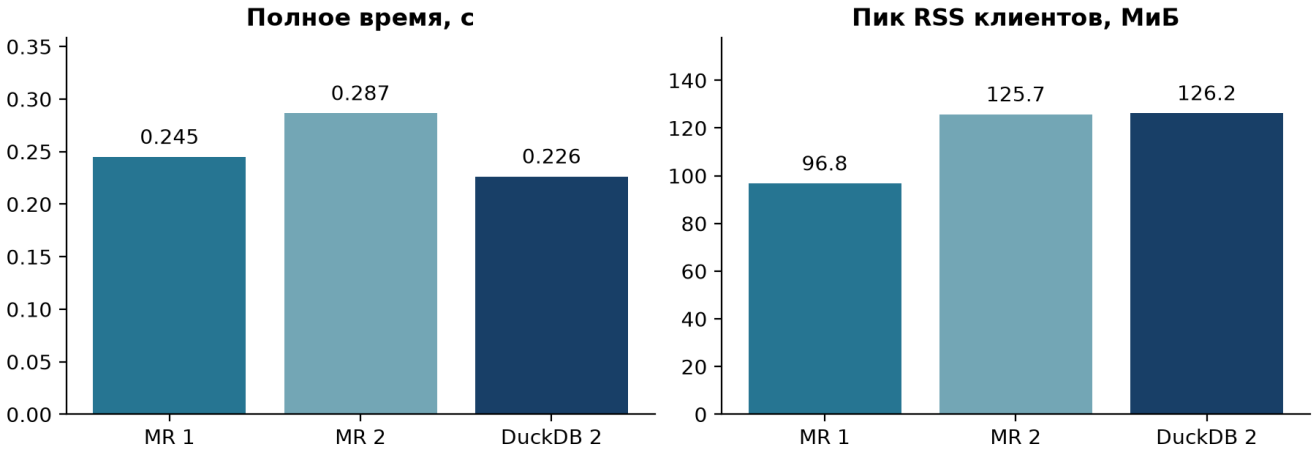
Совпали все 33 456 строк (размер сочетания, список навыков, частота) во всех 15 полных запусках. Дополнительно прошли 7 тестов: ручной пример, случайные множества, повторные навыки, параллельность, PostgreSQL-массивы, UTF-8 на границе чанка и зарплатная база сравнения.

# 3. Время и использование памяти

Каждый запуск выполнен в новом процессе. Измеряется полное время от запуска до выхода, включая импорты, чтение S3, расчёт и запись одной и той же полной таблицы JSON. Отдельно сохранены время основной функции и CPU-время родителя вместе с завершёнными дочерними процессами.

После пилота сделано по 5 запусков трёх конфигураций; порядок методов чередовался. Кеш ОС и RustFS не сбрасывался. Следовательно, это измерения с прогретым хранилищем, а не холодного скачивания. Настройка расширений, подготовка данных, загрузка S3, аналитика и PDF в сравнение методов не входят.

| Метод | Полное время, с | Мин.-макс., с | Пик RSS, МиБ |
|---|---|---|---|
| MapReduce, 1 процесс | 0.245 | 0.239-0.249 | 96.8 |
| MapReduce, 2 процесса | 0.287 | 0.283-0.312 | 125.7 |
| DuckDB, 2 потока | 0.226 | 0.223-0.241 | 126.2 |




На графике и в таблице показаны медианы по пяти запускам. RSS измерялся как сумма резидентной памяти всего дерева клиентских процессов, включая map-процессы; RustFS измерялся отдельно. Пауза между опросами - 5 мс плюс время самого обхода дерева. Короткие пики могут быть пропущены, а общие страницы процессов могут учитываться повторно: это оценка RSS, не PSS.

Максимум наблюдаемого RSS RustFS за всю серию: 106.6 МиБ. Лимит памяти DuckDB - 1 ГиБ; SQL материализует вход и строки сочетаний. Подробный профиль финального GROUP BY сохранён отдельно в *.duckdb-profile.json; он не заменяет замер всего процесса.

На этой выборке DuckDB выполнил полный цикл за 0.226 с, последовательный MapReduce - за 0.245 с, параллельный - за 0.287 с. Два map-процесса здесь не ускорили обработку: их запуск и передача пакетов дороже выигрыша на 710 записях. Разрыв между DuckDB и последовательным вариантом невелик; переносить его на большие данные без нового измерения нельзя.

# 4. Навыки и зарплата: методика

Единица анализа - вакансия, а не упоминание навыка. Для закрытой зарплатной вилки используется середина (from+to)/2; для односторонней - единственная объявленная граница. Это оценка предложенной оплаты, а не фактическая зарплата сотрудника. Пропуски не заменяются нулями; неположительные, нечисловые и обратные вилки исключаются из зарплатных расчётов.

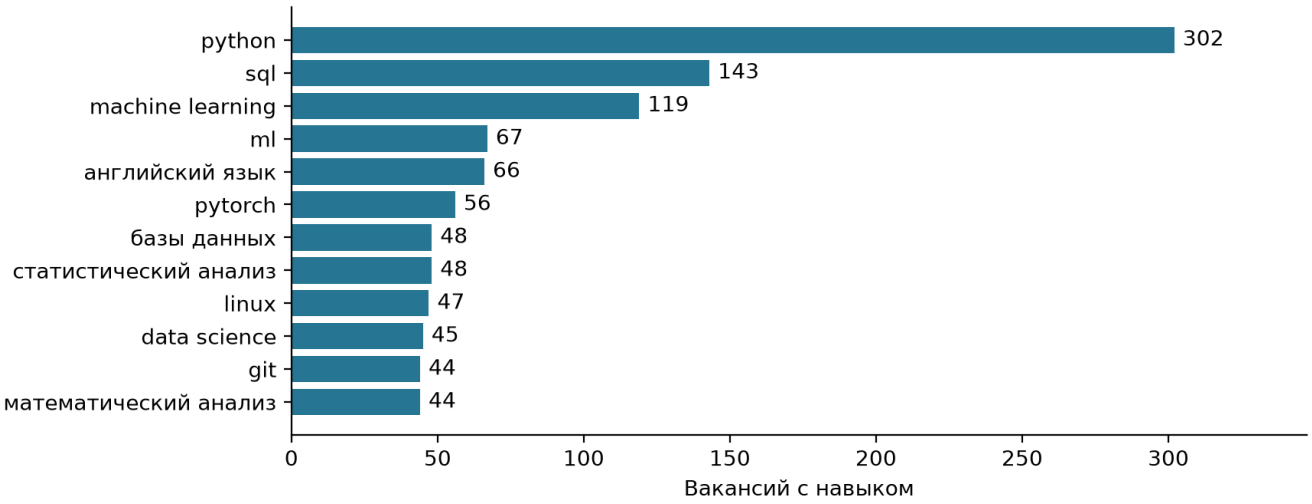
Валюты и зарплаты до/после налога не смешиваются. Основная группа выбрана по максимальному числу вакансий с известными навыками, рублёвой зарплатой и известным налоговым статусом: RUR, gross=false, то есть оплата «на руки». Валютный пересчёт и предположения о ставке налога не применяются.

| Основная зарплатная группа | Значение |
|---|---|
| С известной зарплатой и списком навыков | 58 |
| Медиана группы, рублей в месяц | 200 000 |
| Из них с обеими границами вилки | 36 |
| Минимальный размер каждой сравниваемой группы | 5 |
| Пар и троек, прошедших этот порог | 41 |

Для каждого сочетания сравниваются две непересекающиеся группы одной зарплатной когорты: вакансии, содержащие все навыки сочетания, и остальные вакансии с известным списком навыков. Дополнительные навыки разрешены. Разница = медиана первой группы минус медиана второй; относительная разница = 100 × (медиана первой / медиана второй - 1).

В каждой группе требуется не меньше 5 вакансий. Это исследовательский порог для ограниченной зарплатной выборки, а не гарантия устойчивости. Для отобранных крайних результатов рассчитаны точечные 95% бутстреп-интервалы разницы медиан: 2 000 повторов, seed=20261011. Интервалы не скорректированы на перебор сочетаний и выбор крайних результатов.

### Наиболее частые навыки во всех 710 вакансиях



# 5. Наибольшие различия медиан

Все суммы в таблицах указаны в тысячах рублей в месяц «на руки». n+ - число вакансий с сочетанием, n- - без полного сочетания. Сравнение проведено внутри основной группы из 58 вакансий. Полная таблица из 41 сочетания сохранена в analysis.json.

## Пары с наибольшим превышением медианы

| Сочетание | n+ / n- | Медиана + / - | Разница, % |
|---|---|---|---|
| nlp + pytorch | 7 / 51 | 375 / 150 | +150.0 |
| nlp + python | 7 / 51 | 320 / 150 | +113.3 |
| opencv + python | 5 / 53 | 400 / 200 | +100.0 |

## Тройки с наибольшим превышением медианы

| Сочетание | n+ / n- | Медиана + / - | Разница, % |
|---|---|---|---|
| nlp + python + pytorch | 6 / 52 | 397.5 / 150 | +165.0 |
| python + алгоритмы + машинное обучение | 5 / 53 | 325 / 190 | +71.1 |

## Пары с наименьшей медианой относительно базы

| Сочетание | n+ / n- | Медиана + / - | Разница, % |
|---|---|---|---|
| git + python | 6 / 52 | 112.5 / 200 | -43.8 |
| machine learning + машинное обучение | 7 / 51 | 130 / 200 | -35.0 |
| business development + автоматизация технологических процессов | 5 / 53 | 150 / 200 | -25.0 |

## Тройки с наименьшей медианой относительно базы

| Сочетание | n+ / n- | Медиана + / - | Разница, % |
|---|---|---|---|
| machine learning + python + машинное обучение | 5 / 53 | 130 / 200 | -35.0 |
| business development + автоматизация технологических процессов + бизнес-консультирование | 5 / 53 | 150 / 200 | -25.0 |
| business development + автоматизация технологических процессов + информационные технологии | 5 / 53 | 150 / 200 | -25.0 |

В положительном направлении выделяются NLP + PyTorch и NLP + Python + PyTorch. Отрицательная разница у Git + Python и других сочетаний не означает, что эти навыки вредят доходу: группы могут различаться опытом, работодателем, регионом и характером позиции. Одновременные «machine learning» и «машинное обучение» означают две исходные метки, а не два независимых умения.

# 6. Устойчивость зарплатных выводов

| Сочетание | Разница медиан, тыс. руб. | Точечный 95% интервал, тыс. руб. |
|---|---|---|
| nlp + pytorch | +225 | [120; 325] |
| nlp + python + pytorch | +247.5 | [120; 325] |
| git + python | -87.5 | [-192.5; -20] |
| machine learning + python + машинное обучение | -70 | [-170; 250] |

При повышении порога до 10 наблюдений в каждой группе остаётся только одна пара и ни одной тройки. Следовательно, результаты для троек чувствительны к малой выборке. Нельзя назвать весь рейтинг устойчивым или статистически подтверждённым только на основании этой выгрузки.

Если оставить только закрытые вилки, остаются 36 зарплатных наблюдений и 29 допустимых сочетаний при пороге 5. Для gross=true расчёт выполнен отдельно; эта группа содержит 11 вакансий с навыками. Полные результаты обеих проверок сохранены в analysis.json.

| Сочетание | Все вилки: n / Δ, % | Только закрытые: n / Δ, % |
|---|---|---|
| nlp + pytorch | 7 / +150.0 | 7 / +150.0 |
| nlp + python + pytorch | 6 / +165.0 | 6 / +165.0 |
| git + python | 6 / -43.8 | Не проходит порог 5 |
| machine learning + python + машинное обучение | 5 / -35.0 | Не проходит порог 5 |

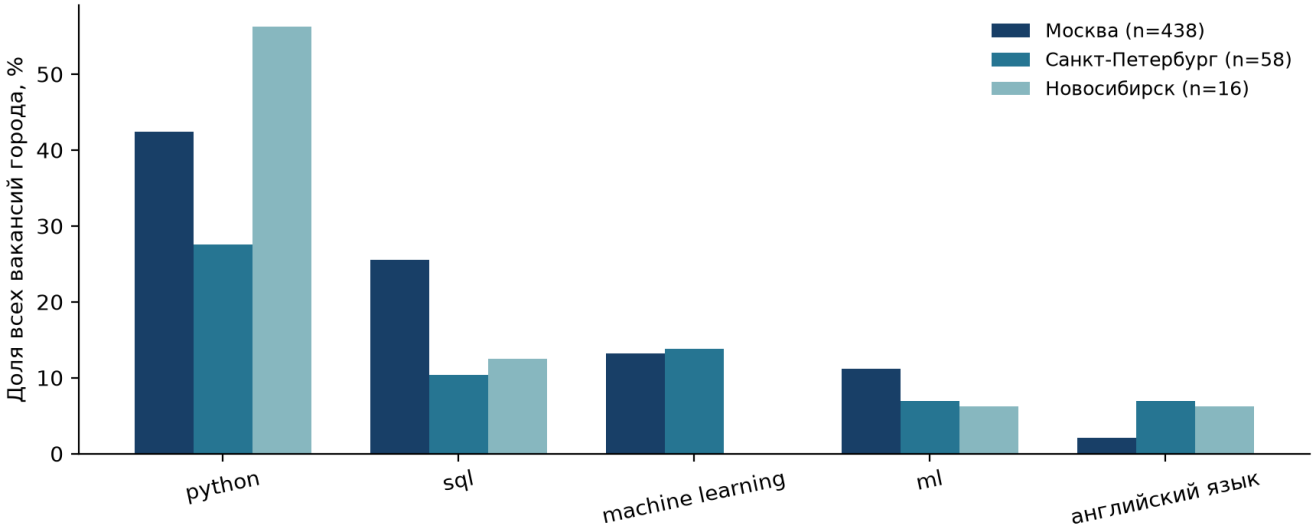
Только 118 из 710 вакансий раскрывают зарплату, а известные навыки и сопоставимая оплата «на руки» одновременно доступны лишь для 58. Пропуски, выборочная публикация зарплат и отсутствие поправок на опыт могут смещать оценку. Работа показывает наблюдаемые различия в исторических объявлениях; причинный эффект конкретного навыка здесь не установлен.

# 7. Сравнение трёх регионов

Выбраны Москва, Санкт-Петербург и Новосибирск - три российских города с наибольшим числом ML/RnD-вакансий в этой выборке. Регион вакансии относится к месту объявления, а не обязательно к месту проживания удалённого сотрудника.

| Город | Всего / с навыками | n зарплат RUR net | Медиана, тыс. руб. | Q25-Q75, тыс. руб. |
|---|---|---|---|---|
| Москва | 438 / 263 | 37 | 250 | 150-320 |
| Санкт-Петербург | 58 / 31 | 8 | 200 | 118.75-352.5 |
| Новосибирск | 16 / 11 | 8 | 137.5 | 97.5-250 |

Для медиан города взяты все вакансии с известной рублёвой зарплатой «на руки», включая объявления без списка навыков; это отличается от зарплатного сравнения сочетаний, где список навыков обязателен. Прочие валюты и gross=true исключены одинаково для трёх городов.




Спрос на навык показан как доля всех вакансий соответствующего города, где навык явно указан, а не как число упоминаний. Разный уровень заполнения списков навыков влияет на доли: у Москвы они известны в 263 из 438 объявлений, у Петербурга в 31 из 58, у Новосибирска в 11 из 16.

Москва лидирует по числу объявлений и медиане раскрытых зарплат. Интерквартильные интервалы городов перекрываются, а для Петербурга и Новосибирска доступны только по 8 сопоставимых зарплат. Поэтому разница медиан описывает эту выборку; её недостаточно для вывода о гарантированной региональной надбавке.

# 8. Воспроизведение и выводы команд

Все команды выполняются из корня репозитория data-engineering-hw3 (локально: homeworks/hw-3). Зависимости закреплены в requirements.lock. bootstrap.py устанавливает httpfs и использует бинарный RustFS из ДЗ 2 либо скачивает проверенный релиз 1.0.1. Повторная подготовка сверяет SHA-256 исходного CSV.

```
uv venv .venv --python 3.12
uv pip install --python .venv/bin/python -r requirements.lock
.venv/bin/python bootstrap.py
.venv/bin/python prepare.py --download
.venv/bin/python -m unittest -v
.venv/bin/python run_experiment.py --repeats 5
.venv/bin/python build_report.py
```

## Фрагменты фактического вывода


```
{
  "method": "mapreduce",
  "rows": 710,
  "bytes": 217135,
  "chunks": 4,
  "map_tasks": 3,
  "core_seconds": 0.11517624999396503,
  "cpu_seconds": 0.11320499999999999
}

{
  "equal": true,
  "compared_runs": 15,
  "rows_in_each_result": 33456,
  "comparison": "Exact equality of every (size, skills, frequency) row; no tolerances",
  "distinct_combinations": {
    "1": 587,
    "2": 5811,
    "3": 27058
  },
  "result_sha256": "df66db9869ab052a2db76cab4cd7c61f5488578d69bba7cca53e903bb144030d"
}
```


Полные выводы: artifacts/prepare.log, pilot.log, run.log, tests.log; измерения: measurements.json; сверка: verification.json; S3: uploads.json. Схема и контрольные суммы: dataset.json. SQL-профили и выводы каждого процесса находятся в artifacts/runs/.

## Источники


[1] Ilya (ilyazawilsiv). IT Vacancies from HeadHunter Website, version 1. Kaggle, Apache 2.0. Страница набора. Период 18.09-17.10.2023.

[2] DuckDB. S3 API Support: httpfs, secrets и прямое чтение объектов. Официальная документация.

[3] RustFS 1.0.1, повторно использованный локальный стенд ДЗ 2. Релиз.


Код, результаты вычислений, профили времени и памяти и PDF опубликованы в публичном репозитории sharingan000/data-engineering-hw3.